Consensus and Raft

agreeing on one history despite failures

Watch the 4-min video

In one line: consensus lets a group of nodes agree on one ordered log of commands even when some crash; Raft elects a leader per term, the leader replicates log entries, and an entry is committed once a majority stores it, so a cluster of 2f+1 nodes survives f failures.

1 Why agreement is hard

Kubernetes stores every pod, service and secret in **etcd**. If that data were lost or inconsistent, the cluster would fall apart, yet any etcd node can crash at any time. How do several machines agree on one history despite crashes, lost messages and partitions? With a **consensus** algorithm. Raft (2014) was designed to be understandable; it powers etcd, Consul, CockroachDB, TiKV and Kafka's KRaft mode.

2 The goal: a replicated state machine

Every node keeps a **log** of commands. If all nodes apply the same commands in the same order from the same starting state, they reach the same state. Consensus is the problem of agreeing on that log.

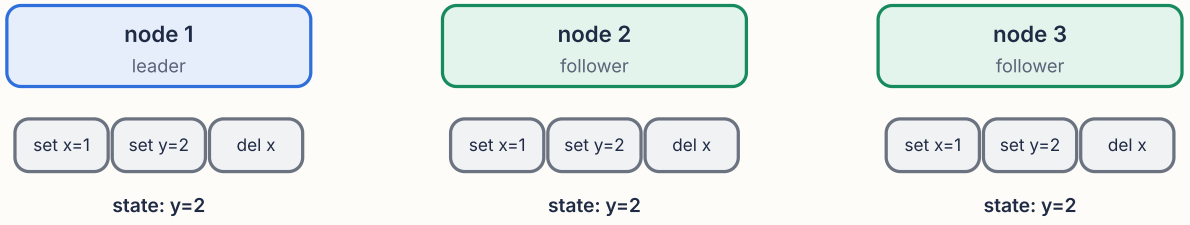


Figure 1. Raft replicates a log; each node applies it to its own copy of the state.

3 Terms and leader election

Raft has one **leader**; the others are **followers**. Time is divided into numbered **terms**, each with at most one leader. The leader sends regular heartbeats. A follower that hears nothing for a **randomised election timeout** (e.g. 150 to 300 ms) becomes a **candidate**: it increments the term, votes for itself and requests votes. Each node votes at most once per term, and only for a candidate whose log is at least as up to date as its own. A majority of votes makes it leader. Random timeouts make split votes rare; if one happens, a new term starts.

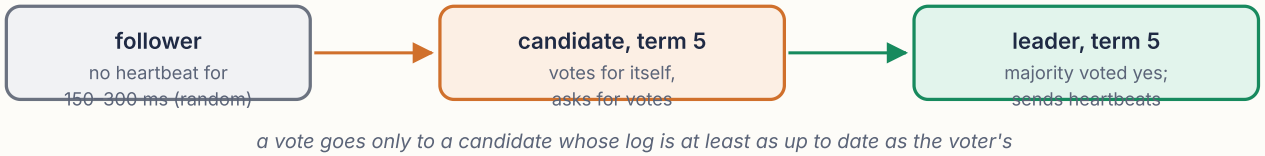


Figure 2. From follower to leader. The up-to-date-log rule guarantees the new leader already holds every committed entry.

4 Log replication and commit

Clients send commands to the leader. The leader appends the entry to its log and sends `AppendEntries` to followers. Once a **majority** (including itself) has stored the entry, it's **committed**: the leader applies it, replies to the client, and tells followers to apply it too. Any future leader must have a majority's votes, and any two majorities overlap, so a committed entry can never be lost.

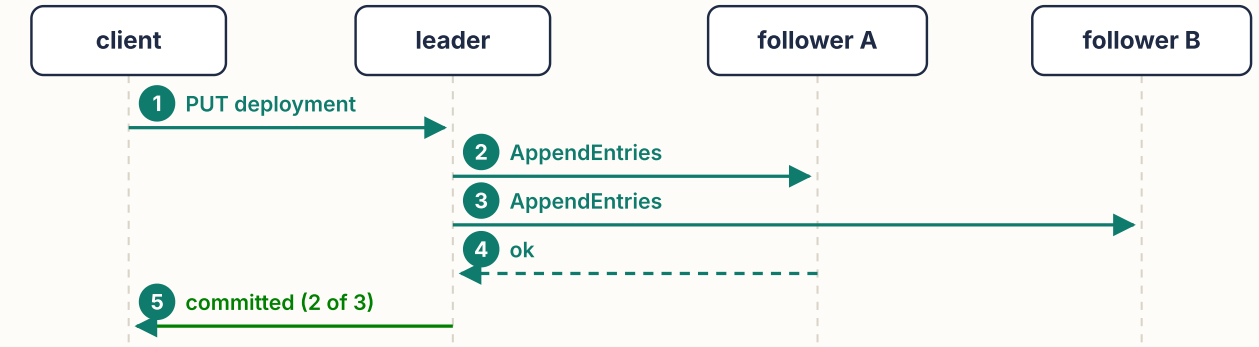


Figure 3. A write commits when 2 of 3 nodes store it; the third can catch up later.

WORKED EXAMPLE: A THREE-NODE ETCD

The Kubernetes API server writes a new deployment to the etcd leader. The leader appends it to its log and replicates it. Follower A acknowledges within a millisecond; follower B is slow. With 2 of 3 stored, the entry commits and the API server gets its answer, without waiting for B. If the leader then crashes, A or B times out, starts term 6, and the up-to-date-log rule makes sure the winner has the deployment. The cluster tolerates **one** failure; with five nodes it tolerates two.

5 Practical points

- Cluster sizes are odd: 3 or 5 (rarely 7). More nodes add fault tolerance but slow writes.
- Every write costs a round trip to a majority, so spread nodes across zones, not distant regions, unless you accept the latency.
- Logs are compacted into **snapshots**; membership changes use joint consensus or single-node changes.
- Reads must also be careful: a leader confirms it's still leader (read index or lease) before serving linearizable reads.

COMMON MISTAKES

- Running two or four nodes: no added tolerance over one or three.
- Spreading a Raft group across continents and wondering why writes take 200 ms.
- Serving reads from any follower and calling it strongly consistent.
- Using etcd as a general-purpose database for large data.

6 Key terms

Consensus: nodes agree on one value or log despite failures

Term: numbered period with at most one leader

Committed: stored on a majority; can't be lost

Replicated state machine: same log applied everywhere

Election timeout: random wait before a follower runs for leader

Quorum: majority: 2 of 3, 3 of 5

7 Interview questions

Q1. What problem does Raft solve? BASIC

It keeps a replicated log consistent across nodes despite crashes and lost messages, so the nodes act as one reliable state machine.

Q2. When is an entry committed in Raft? BASIC

When the leader has stored it on a majority of nodes in its current term; then it's safe to apply and acknowledge.

Q3. How many failures does a 5-node Raft cluster tolerate, and why not use 6? SCENARIO

Two. A 6-node cluster needs 4 for a majority and still tolerates only two failures, while every write waits for one more node.

Q4. The leader is partitioned from the other two nodes. What happens? SCENARIO

The two form a majority, elect a new leader in a higher term and keep committing. The old leader can't reach a majority, so its writes never commit, and it steps down when it sees the higher term.

Q5. Why are election timeouts randomised? TRICKY

So followers don't all become candidates at once and split the vote forever; one usually times out first and wins.

Q6. Can a Raft leader serve reads from its own state without talking to anyone? TRICKY

Only if it's sure it's still leader. Otherwise a deposed leader could return stale data. Raft uses a read index (a heartbeat round to a majority) or a time-based leader lease.

Next: SD-H-010 · Paxos Overview backendroadmap.com